

Graphics for Categorical Variables

STAT 200 – Introductory Statistics

Torin Quinlivan

January 12, 2025



Today's Objective

After today's lecture, you should:

- create the following univariate graphics for categorical variables:
 - pie chart
 - bar chart
- create the following bivariate graphics for categorical variables:
 - side-by-side bar chart
 - stacked bar chart
 - mosaic plot
- determine which graphic(s) are appropriate to tell the story of your data
- interpret the presented graphics of others
- modify graphics to make them presentation-worthy

Reading in the Data

To read into R the data we will use for this lecture, run the following code:

```
dt = read.csv("http://rfs.kvasaheim.com/data/crime.csv")
```

This will read the dataset in from Dr. Forsberg's website.

If the .csv file is on your computer, use `read.csv("file path")`, i.e.

```
dt = read.csv("/Users/tquinlivan/R/crime.csv")
```

`read.csv` reads the .csv file into as a data frame. If you want the columns attached as individual vectors, use the following function:

```
attach(dt)
```

Three Graphics Packages

There are three main packages to use while plotting in R:

- `base` (This is the built-in package in R.)
- `grid`
- `ggplot`

They all use a different way of thinking about how graphics should be created. `grid` and `ggplot` are packages that can be added to base R, and can make prettier plots.

How to Think About Graphics

We will discuss the base R functions in this lecture.

How should we think about our process for creating plots?

1. Plan your “painting”.
2. Define the parameters of the “canvas”.
3. Start the “canvas”.
4. Add to the “canvas”.

The Basic Categorical Graphics

Some basic **univariate** categorical graphics consist of:

- Pie charts
- Bar charts

Some basic **bivariate** categorical graphics consist of:

- Side-by-side bar charts
- Stacked bar charts
- Spine charts
- Mosaic plots

Pie Chart

You're all familiar with pie charts. It is a circle split into segments of the size of the proportion of the sample in each category.

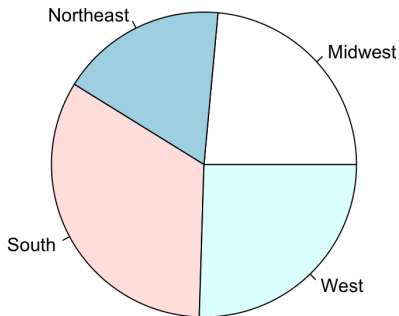
Be careful with pie charts. They can be hard to read and easy to misuse.

The function in R to make a pie chart is `pie(table(var))`.

The `table` function returns the count of each element in a vector.

Pie Chart

```
pie(table(census4))
```



Bar Chart

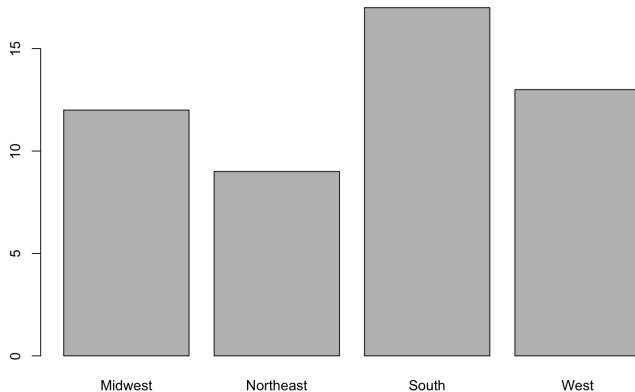
You know bar charts as well. It is a graph with bars whose height represents the frequency of each category.

Often bar charts The function in R to make a bar chart is

`barplot(table(var)).`

Bar Chart

```
barplot(table(census4))
```



Bivariate Graphics

The previous two kinds of charts help break down the distribution of a single variable.

What if we want to compare the distribution of two variables together?

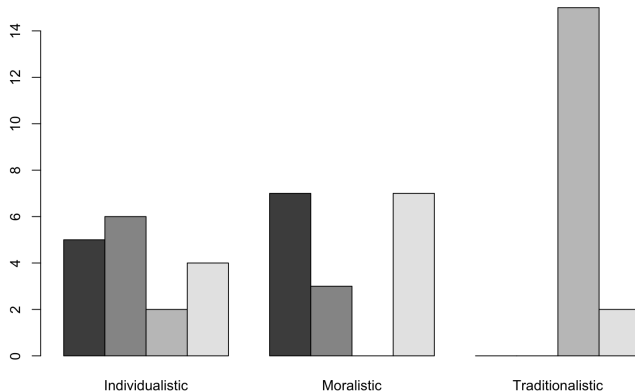
Side-by-Side Bar Chart

A side-by-side bar chart essentially makes a bar chart of one variable for each category of the other variable.

The function in R to make a side-by-side bar chart is `barplot(table(var1, var2), beside=TRUE)`.

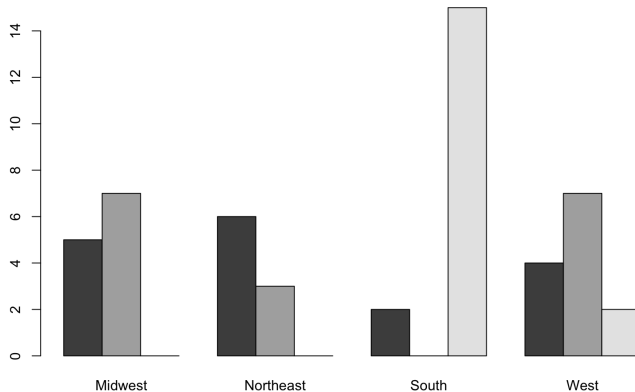
Side-by-Side Bar Chart

```
barplot(table(census4, domPolCulture), beside=TRUE)
```



Side-by-Side Bar Chart

```
barplot(table(domPolCulture, census4), beside=TRUE)
```



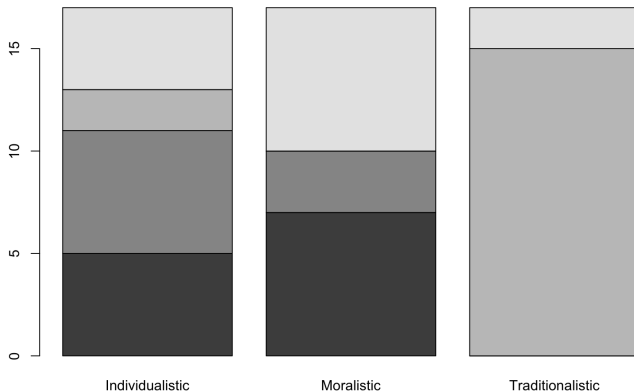
Stacked Bar Chart

A side-by-side bar chart makes a bar chart of one variable and colors in the segments of those bars according to the distribution of the other variable.

The function in R to make a side-by-side bar chart is `barplot(table(var1, var2))`.

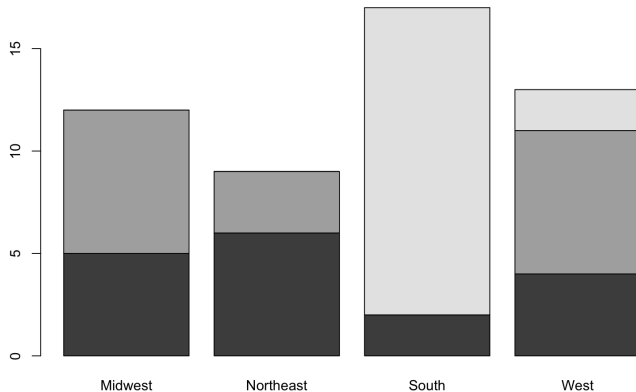
Stacked Bar Chart

```
barplot(table(census4, domPolCulture))
```



Stacked Bar Chart

```
barplot(table(domPolCulture, census4))
```



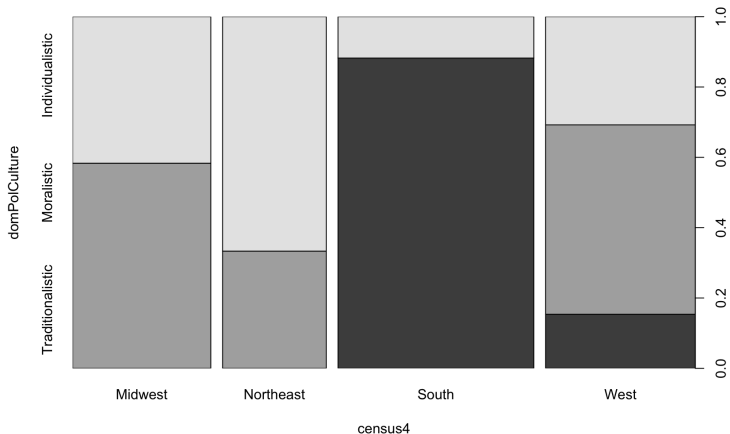
Spine Chart

A spine chart is a stacked bar chart where instead of the height of the bar reflecting the frequency, but the width.

The function in R to make a side-by-side bar chart is `spineplot(table(var1, var2))`.

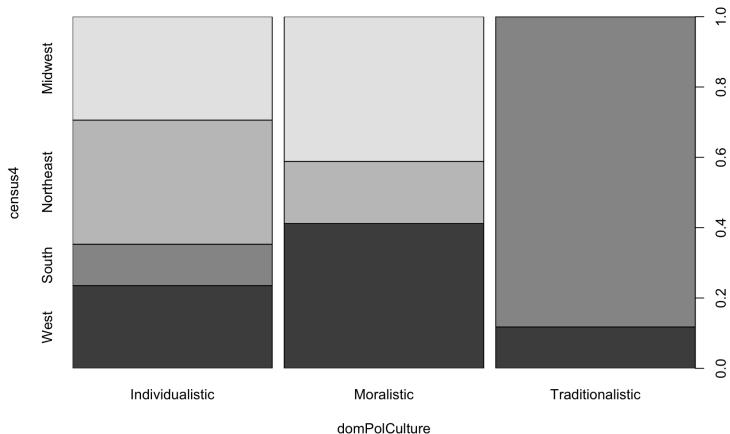
Spine Chart

```
spineplot(table(census4, domPolCulture))
```



Spine Chart

```
spineplot(table(domPolCulture, census4))
```



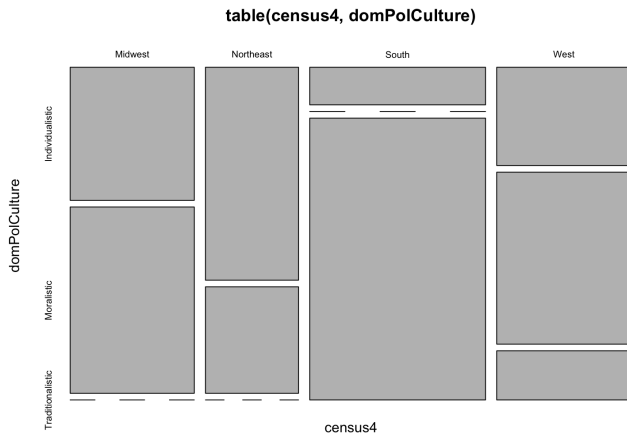
Mosaic Chart

A mosaic chart does the same as the spine chart, but has separate boxes for the categories in the bars.

The function in R to make a side-by-side bar chart is `mosaicplot(table(var1, var2))`.

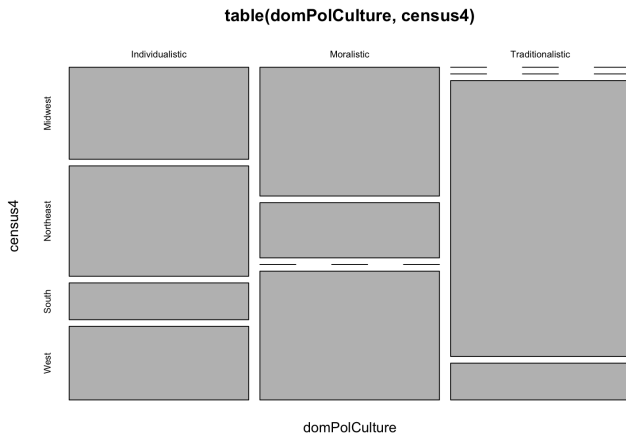
Mosaic Chart

```
mosaicplot(table(census4, domPolCulture))
```



Mosaic Chart

```
mosaicplot(table(domPolCulture, census4))
```



Planning Your Graphic

What we've done so far is the default settings of each plot type. We can customize our plots more, if we want to.

Here are some things to think about before starting your graphic:

- what data should be illustrated by the where will the data be placed on the canvas?
- what margins will you use?
- what font? should it differ between labels and values?
- what sizes should the fonts be? should they differ between labels and values?
- should the axes have additional space?
- what should the value orientation be?

Note: It is very helpful to search the Internet for ideas for your graphic. There are many, *many* resources available.

Planning Your Graphic

Now that you have thought about what your graphic will look like (the planning), let's formally set the parameters of the graphic.

This code sets the margins to be 4 lines at the bottom, 4 at the left, 1 at the top, and 1 at the right:

```
par(mar=c(4,4,1,1))
```

This code specifies the font family is serif, with labels bolded and values italicized:

```
par(family="serif", font.lab=2, font.axis=3)
```

This code specifies the labels are 20% larger and values are 20% smaller than the base size:

```
par(cex.lab=1.2, cex.axis=0.8)
```

Planning Your Graphic

Now that you have thought about what your graphic will look like (the planning), let's formally set the parameters of the graphic.

This code requires the axes to meet at (0, 0):

```
par(xaxs="i", yaxs="i")
```

This code forces all labels to be horizontal (for readability):

```
par(las=1)
```

This code makes the next 6 plots appear together in one image as a 2×3 matrix:

```
par(mfrow=c(2,3))
```

Starting a New Window

The following code starts the graphic and sets the viewing window to $(0, 10) \times (0, 100)$.

```
plot.new()  
plot.window(xlim=c(0,10), ylim=c(0,100))
```

This will start a generic window. Some graphing functions have their own “starting” functions. Those we are covering in this section do. So, the above two lines are not needed for those functions.

Starting a New Window

To create a bar chart with some interesting colors, let's use this code:

```
barplot( table(census4),  
         col=c("red4", 3, "#291999", rgb(0.5,0.5,0.5)) )
```

R color cheat sheet: <https://www.nceas.ucsb.edu/sites/default/files/2020-04/colorPaletteCheatsheet.pdf>

Annotating a Graph

The following code cleans up the graphic by adding label for the variables and adds a base line.

```
title(xlab="Census Region", line=2.75)
title(ylab="Count of States in Region", line=2.75)
abline(h=0, lwd=2)
```

Fancy Bar Chart

Here's our fancy new bar chart after running all that code:

